Inference of Vortex Shedding using MeshGraphNet Architecture

Arturo Burgos
Mechanical Science & Engineering
University of Illinois Urbana Champaign

Problem

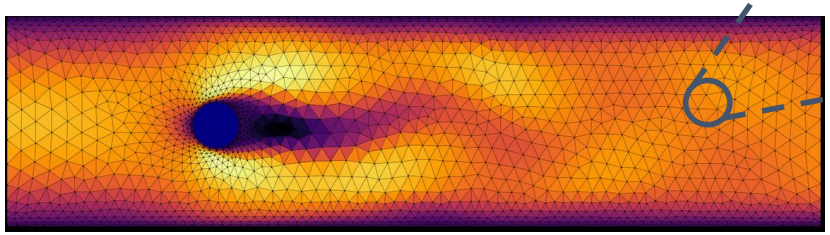
Published as a conference paper at ICLR 2021

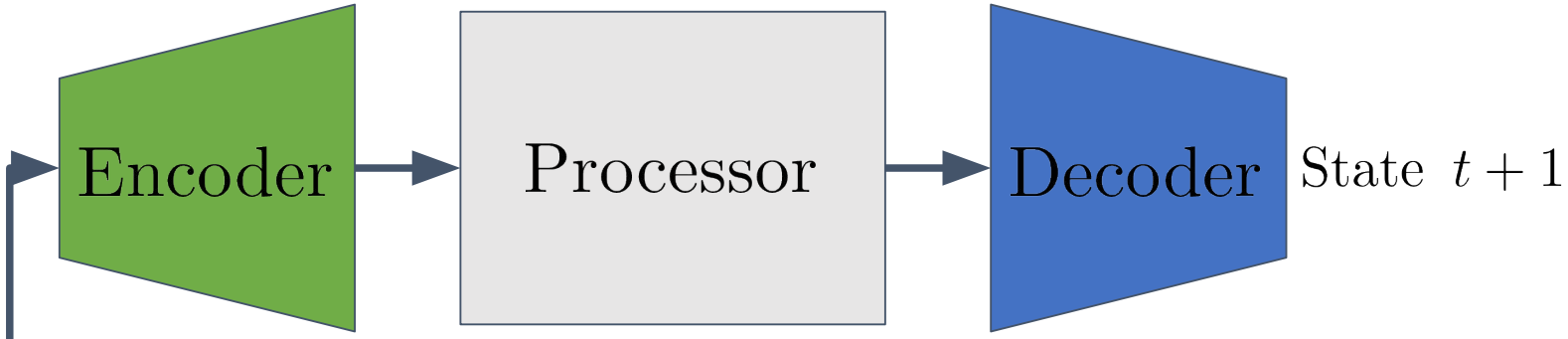
Learning Mesh-Based Simulation with Graph Networks

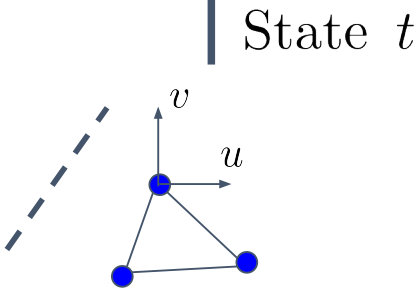
Tobias Pfaff,* Meire Fortunato,* Alvaro Sanchez-Gonzalez,* Peter W. Battaglia
Deepmind, London, UK
{tpfaff,meirefortunato,alvarosg,peterbattaglia}@google.com

Abstract

Mesh-based simulations are central to modeling complex physical systems in many disciplines across science and engineering. Mesh representations support powerful numerical integration methods and their resolution can be adapted to strike favorable trade-offs between accuracy and efficiency. However, high-dimensional scientific simulations are very expensive to run, and solvers and parameters must often be tuned individually to each system studied. Here we introduce MeshGraphNets, a framework for learning mesh-based simulations using graph neural networks. Our model can be trained to pass messages on a mesh graph and to adapt the mesh discretization during forward simulation. Our results show it can accurately predict the dynamics of a wide range of physical systems, including aerodynamics, structural mechanics, and cloth. The model's adaptivity supports learning resolution-independent dynamics and can scale to more complex state spaces at test time. Our method is also highly efficient, running 1-2 orders of magnitude faster than the simulation on which it is trained. Our approach broadens the range of problems on which neural network simulators can operate and promises to improve the efficiency of complex, scientific modeling tasks.





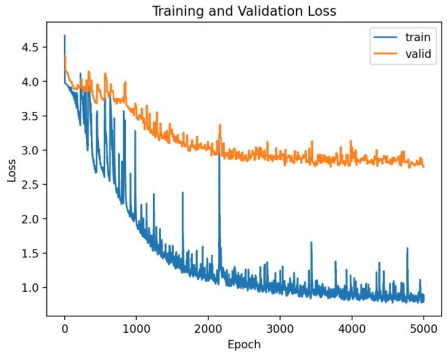



```python
def message(self, x_i, x_j, edge_attr):
    """
    source_node: x_i has the shape of [E, in_channels]
    target_node: x_j has the shape of [E, in_channels]
    target_edge: edge_attr has the shape of [E, out_channels]

    The messages that are passed are the raw embeddings. These are not processed.
    """
    updated_edges = torch.cat(
        [x_i, x_j, edge_attr], dim=1
    )  # tmp_emb has the shape of [E, 3 * in_channels]
    updated_edges = self.edge_mlp(updated_edges) + edge_attr
    return updated_edges
```

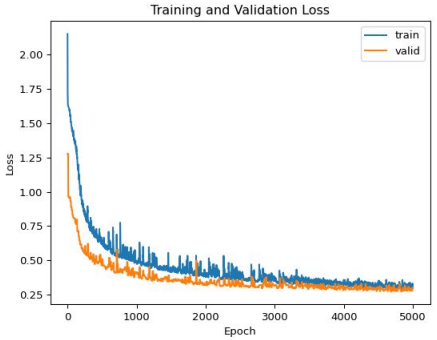
```python
def aggregate(self, updated_edges, edge_index, dim_size=None):
    """
    First we aggregate from neighbors (i.e., adjacent nodes) through concatenation
    then we aggregate self message (from the edge itself). This is streamlined
    into one operation here.
    """
    # The axis along which to index number of nodes.
    node_dim = 0
    out = torch_scatter.scatter(
        updated_edges, edge_index[0, :], dim=node_dim, reduce="sum"
    )
    return out, updated_edges
```

Mesh GNNs (MeshGraphNet) Loss function

1st Loss history



2nd Loss history



$$\mathcal{L} = \sqrt{\frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \|\tilde{y}_i - \hat{y}_i\|_2^2}$$

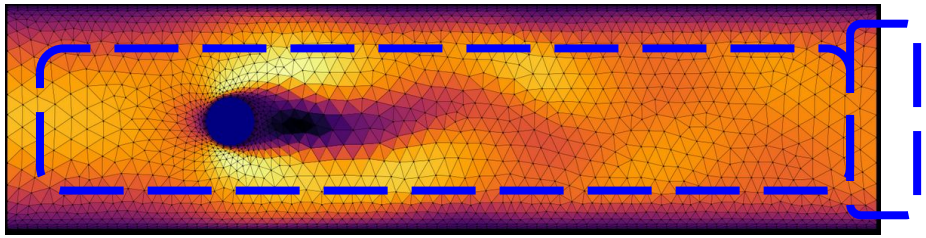
$\mathcal{M}$ is the set of nodes used in the Loss

$\tilde{y}_i$ is the normalized true target at node i

$\hat{y}_i$ is the predicted target at node i

```python
def loss(self, pred, inputs, mean_vec_y, std_vec_y):

    # Define  the node types we calculate the loss for.
    # Since our model has 4 node types (inlet, outlet, wall, internal)
    # and the data is was simulated from a prescribed inlet velocity and no-slip
    # boundary conditions at the walls, we only calculate the loss for the internal
    # and oulet nodes, as the inlet and wall nodes have prescribed dynamics.

    # normal means internal fluid nodes.

    normal = torch.tensor(0)
    outflow = torch.tensor(5)

    # Get the loss mask for the normal and outflow nodes
    loss_mask = torch.logical_or((torch.argmax(inputs.x[:, 2:], dim=1) == normal),
                                 (torch.argmax(inputs.x[:, 2:], dim=1) == outflow))

    # Normalize true values with dataset mean and std
    labels = norm.normalize(inputs.y, mean_vec_y, std_vec_y)

    # Sum of square errors for each node (in here considering all the nodes))
    error = torch.sum((labels - pred) ** 2, axis=1) # compute the error for each node, error is [num_nodes]

    # Root mean squared error for the nodes we are interested in (normal and outflow nodes)
    loss = torch.sqrt(torch.mean(error[loss_mask]))

    return loss
```

The loss seems reasonable considering that size of my dataset (coarse). I plan to add more time-steps to the training phase.



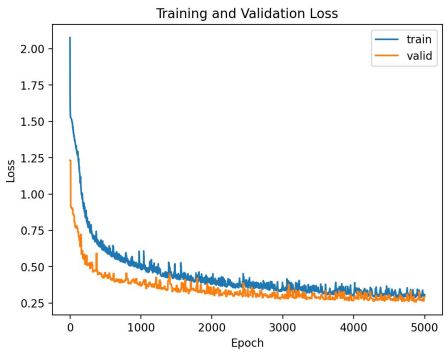
$\mathcal{M} = \text{normal+outflow}$

The biggest challenge so far was identifying why the loss was that high as in the first pass, this is due a mistaken I made in implementing the normalization functionality. There I was simply redefining quantities as mean_vec_x, not summing the normalization for each term (node and edge) in the graph.

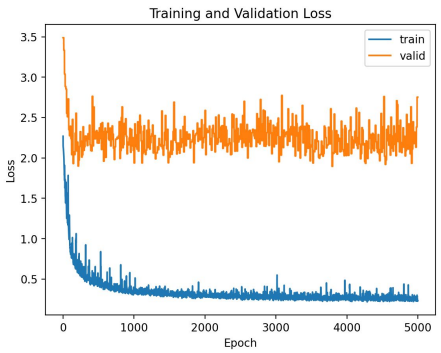
As the next steps I plan to implement wandb log, rollout capabilities, deploy this in the university cluster to increase my training data.

Mesh GNNs preliminary results

1st smoke-test Loss history



2nd smoke-test Loss history



Total Dataset: 1200 trajectories with 600 time-step each

- 1000 trajectories as training
- 100 trajectories as valid
- 100 trajectories as test

First smoke-test:

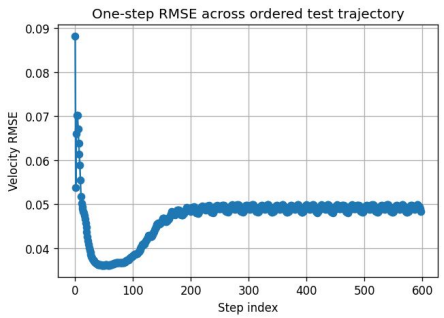
- split train into train & valid sets
  - 45 time-steps train
  - 10 time-steps valid

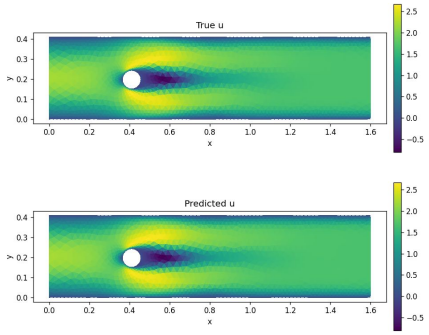
Second Smoke test:

- 500 time-steps of 3 trajectory as train
- 50 time-steps of 1 trajectory as valid
- also 1 trajectory for test

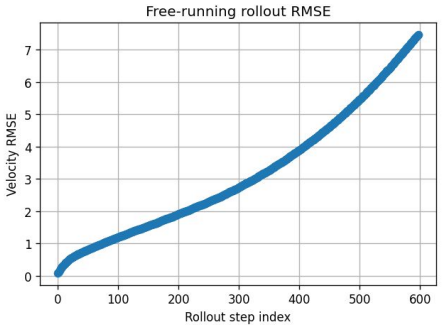
One-step vs. Rollout:

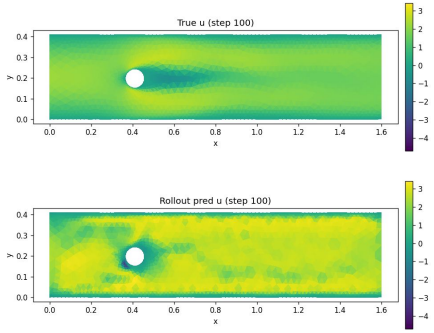
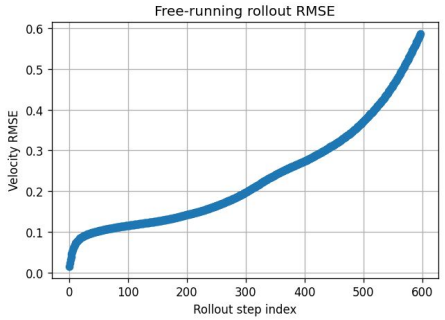
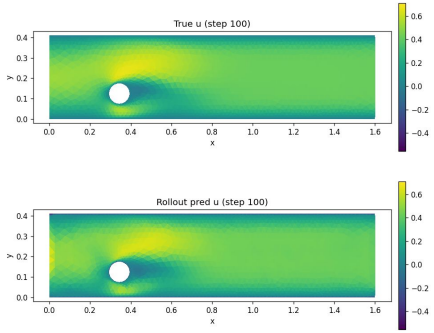
One-step test data RMSE





rollout for test data RMSE





rollout for train data RMSE